GROUP CODE · SURGE 090

Progress Review (Design & Development)

Smart Snake Robot

with Dynamixel Smart Servo Motors

INSTITUTION

Indian Institute of Technology, Delhi

PROJECT GUIDE

Professor Amartansh Dubey

Shashank Jangir

ENTRY NO.

2024EE11048

Bhavesh Bansiwal

ENTRY NO.

2024ME10487

Mahima Chotiya

ENTRY NO.

2024ME11187

PROJECT REPOSITORY

https://github.com/shashankjangir/SurGe-090-Smart-snake-robot

ACTUATOR SELECTION STUDY

Why Dynamixel Motors?

THE REQUIREMENT

A snake robot drives many joints at once. The actuator must deliver:

- Precise position control
- Simple wiring architecture
- Scalability for multiple joints
- Reliable communication
- Real-time feedback
- Compact size & sufficient torque

OPTIONS COMPARED

Criterion	Conventional Servo	Dynamixel Smart Servo
Communication	Individual signal wire per motor	Daisy-chain communication
Wiring Complexity	High	Low
Feedback	Not available	Position, load, temperature, voltage feedback
Controller	External controller required	Built-in controller
Scalability	Difficult for many joints	Designed for multi-joint robots
Precision	Moderate	High
Robotics Applications	Hobby projects	Research and industrial robotics





ARCHITECTURE & DECISION

Daisy-Chain Architecture & Final Selection

Conventional Servo — Separate Wiring



Every motor needs its own signal wire back to the controller.

Dynamixel — Daisy-Chain



One cable carries power and data to every joint in sequence.

KEY LEARNINGS

- Daisy-chain wiring sharply cuts cabling and assembly effort.
- A controller built into every motor simplifies integration.
- Real-time feedback enables future closed-loop control.
- Compact form factor suits modular robot segments.
- High precision and smart control fit advanced robotics.

FINAL SELECTED ACTUATOR

Dynamixel XL330 Series

Chosen for its smart serial communication, full-state feedback, compact form factor and effortless multi-joint scaling — the right fit for a multi-segment snake robot.

GOALS & EXPECTED OUTCOMES

What This Project Sets Out to Build

01

Modular Architecture

A repeatable segment-based mechanical design.

02

Dynamixel Actuators

Integrate smart servos with daisy-chained control.

03

Raspberry Pi Control

A single-board-computer control system.

04

Locomotion Algorithms

Implement serpentine and gait-based motion.

05

Sensor Integration

Add sensing for future intelligent navigation.

06

Scalable Platform

Keep it maintainable and easy to extend.

WORK COMPLETED SO FAR

Quarter-One Progress Summary

OVERALL COMPLETION(till now)

25 %

Phase 1 complete · Phase 2 underway

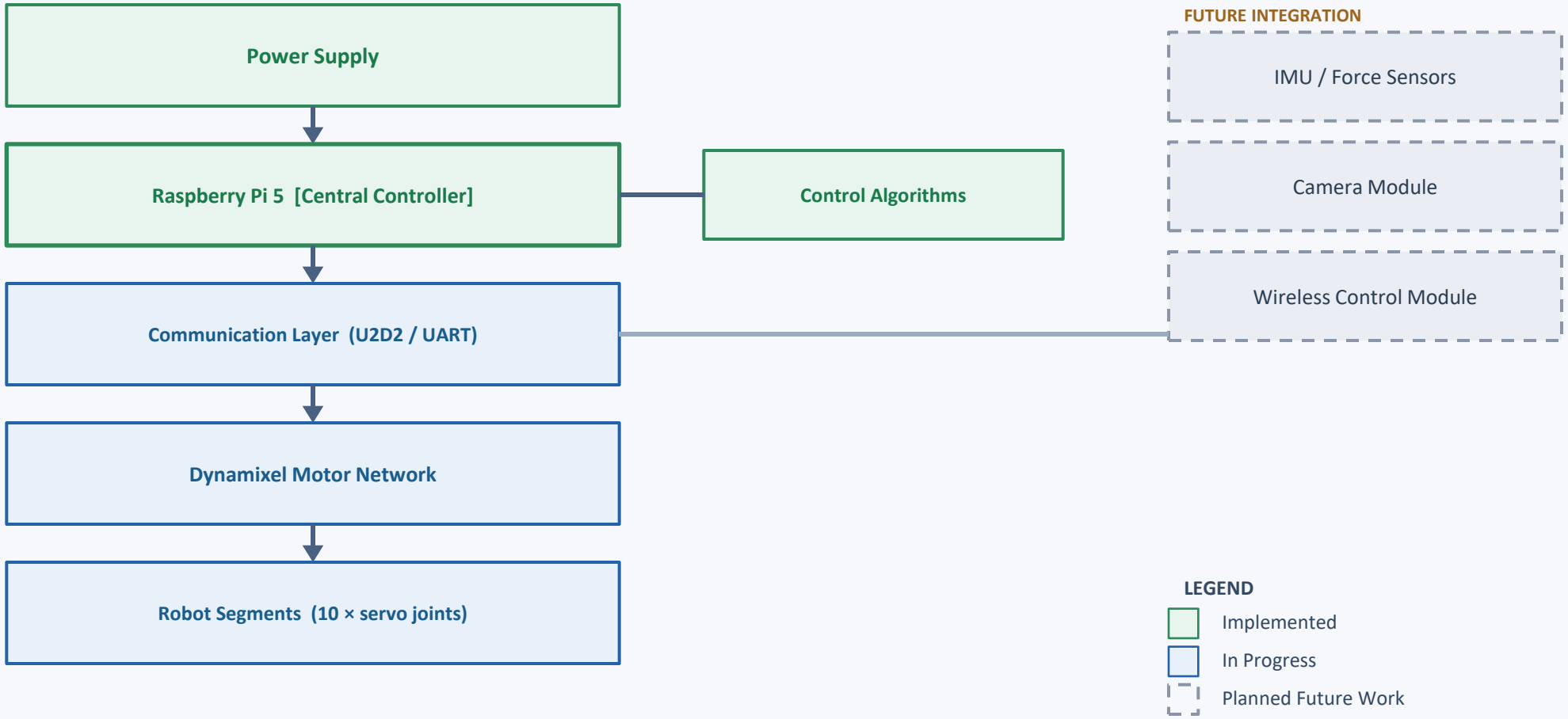
10	2	1
Tasks done	Segments printed	Pi configured

COMPLETED TASKS

- Literature review & project understanding
- Raspberry Pi setup & configuration
- Hardware selection & BOM preparation
- Wi-Fi connectivity testing on the Pi
- CAD design of snake-robot segment
- Basic Raspberry Pi lab projects executed
- Assembly sized to Dynamixel XL330 dims
- GitHub repository created & maintained
- 3D printing of two prototype segments
- Procurement process initiated

Conceptual Block Diagram

Architecture finalized · Implementation in progress



Current Implementation Status

- Raspberry Pi 5 configured and tested
- Communication architecture defined
- Mechanical segments fabricated
- Dynamixel integration pending hardware

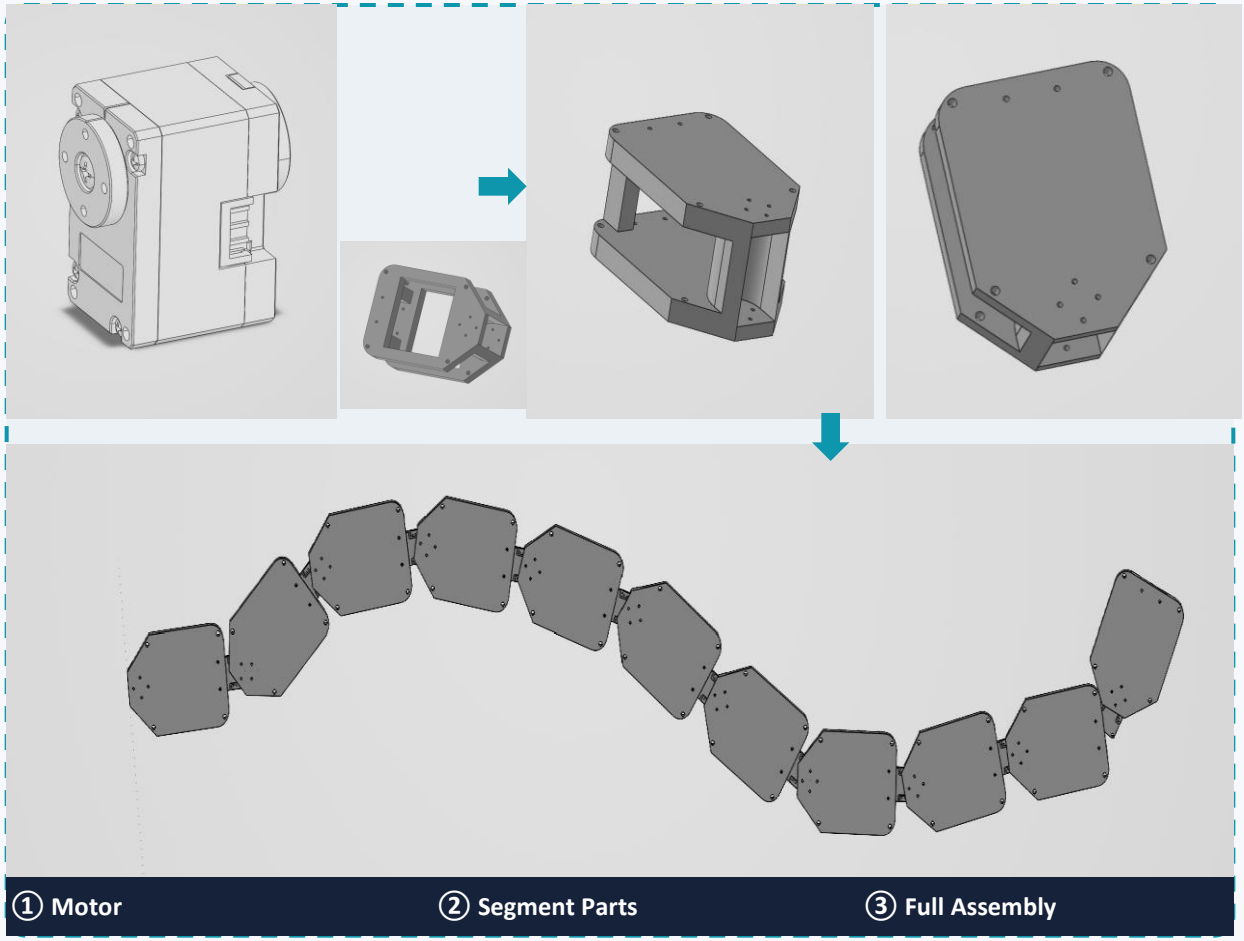
CAD DESIGN DEVELOPMENT

A Segment Designed Around the Motor

The CAD model is built to the exact dimensions of the Dynamixel XL330-M288 servo, using a modular, segment-based architecture focused on manufacturability and scalability.

DESIGN CONSIDERATIONS

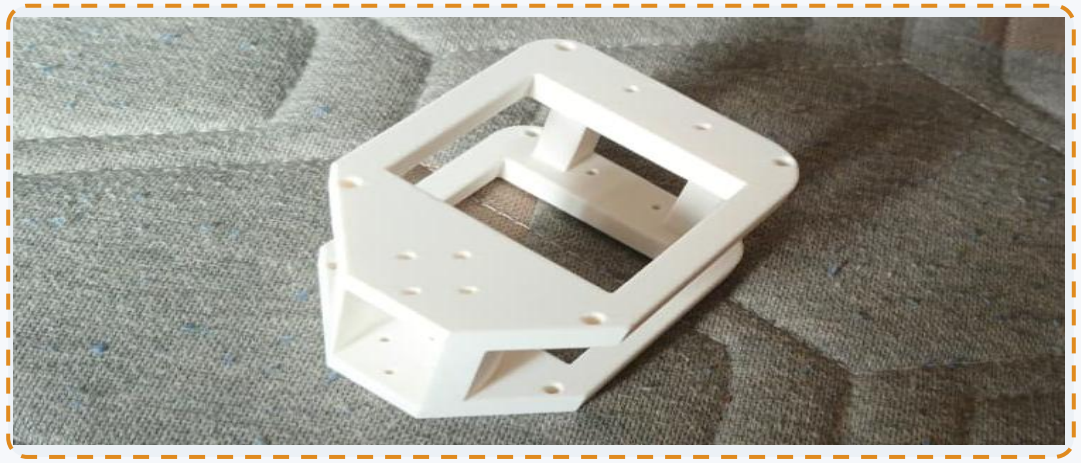
- **Motor mounting** — Precise seats for XL330 servo bodies.
- **Structural rigidity** — Stiff frame to resist motion loads.
- **Cable routing** — Clean daisy-chain channels for wiring.
- **Electronics ready** — Space reserved for future boards.



PROTOTYPE FABRICATION

From Model to Printed Hardware





FABRICATION STATUS

Two prototype segments 3D printed — DONE

Both segments produced from the CAD model.

Initial dimensional verification done — DONE

Printed parts measured against design specs.

Physical validation of CAD initiated — NEXT

Checking fit, tolerances and assembly.

Preparing for motor-integration tests — NEXT

Next: seat XL330 servos into segments.

RASPBERRY PI LEARNING & DEVELOPMENT

Standing Up the Compute Platform

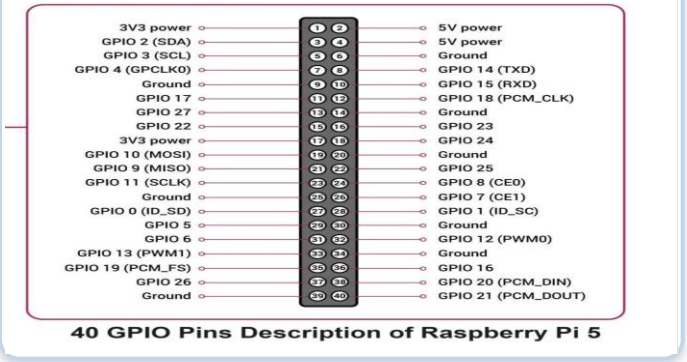
PROGRESS MILESTONES

- Raspberry Pi 5 received & configured ✓
- Raspberry Pi OS installation completed ✓
- Wi-Fi connectivity established ✓
- SSH & remote access explored ✓
- Basic GPIO & intro projects performed ✓
- Linux environment & Python familiarity ✓

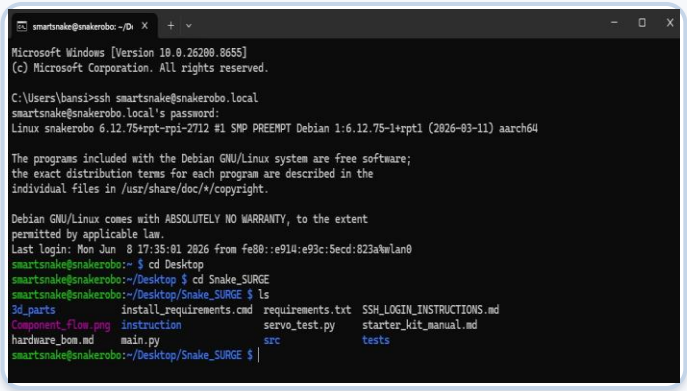
Development Environment Setup



Raspberry Pi OS Desktop



GPIO / Python Testing



SSH Remote Access



System Configuration

Raspberry Pi Platform Successfully Configured and Tested

Hardware Procurement & Resource Availability

Hardware Strategy: Raspberry Pi 4 was planned in the BOM; however, a Raspberry Pi 5 was already available in the laboratory, enabling immediate software development and testing without waiting for delivery. To accelerate development, the team utilised the laboratory Raspberry Pi 5 platform while the ordered components remain under procurement.

AVAILABLE & IN USE

- Raspberry Pi 5 (Laboratory Resource)
- Raspberry Pi Starter Kit
- Raspberry Pi OS Environment
- GitHub Repository
- 3D Printed Prototype Segments

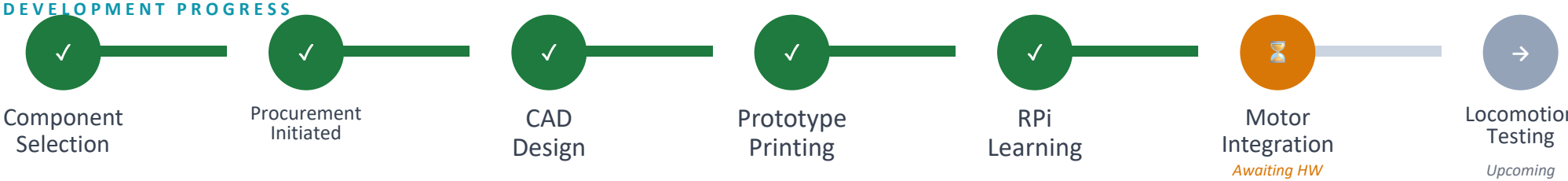
ORDERED — AWAITING DELIVERY

- Dynamixel XL330 Smart Servo Motors
- U2D2 Communication Interface
- U2D2 Power Hub Board
- Dynamixel Communication Cables
- 5V Power Supply
- ESP32 Development Board

COMPLETED PREPARATORY WORK

- Hardware selection finalised
- Bill of Materials prepared
- CAD model completed
- Segment fabrication initiated
- Software environment configured

DEVELOPMENT PROGRESS



★ **Key Achievement:** Although the primary Dynamixel hardware has not yet arrived, the team has successfully utilised available laboratory resources to complete design, fabrication, documentation, and software preparation activities, ensuring project progress is maintained.

REPOSITORY & DOCUMENTATION

Version Control From Day One

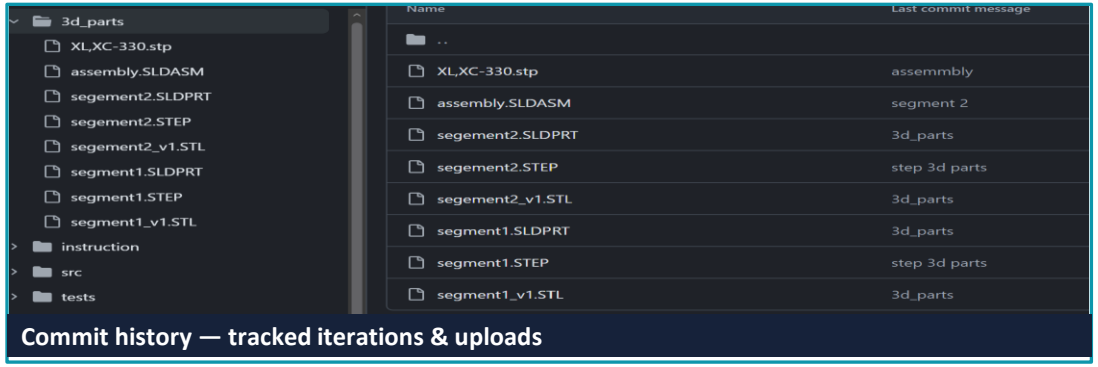
1. **Repository created**
   Central home for the whole project.
2. **Version control established**
   Tracked, reversible change history.
3. **CAD files uploaded**
   Design source committed to the repo.
4. **Design iterations documented**
   Every revision captured in commits.
5. **Progress tracking in place**
   Milestones and issues logged.
6. **Team collaboration workflow**
   Shared branches and pull requests.

✓ All milestones tracked in GitHub

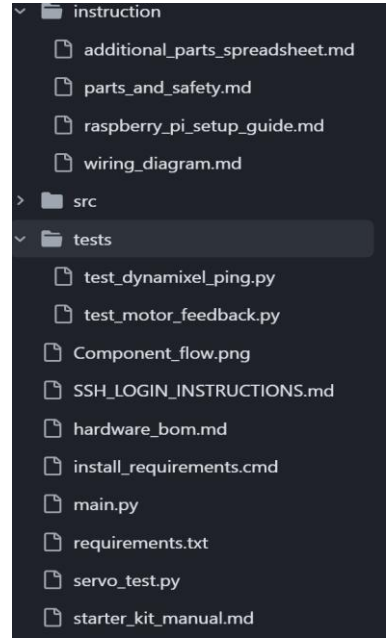
GITHUB REPOSITORY SCREENSHOTS



GitHub repository — project root & structure

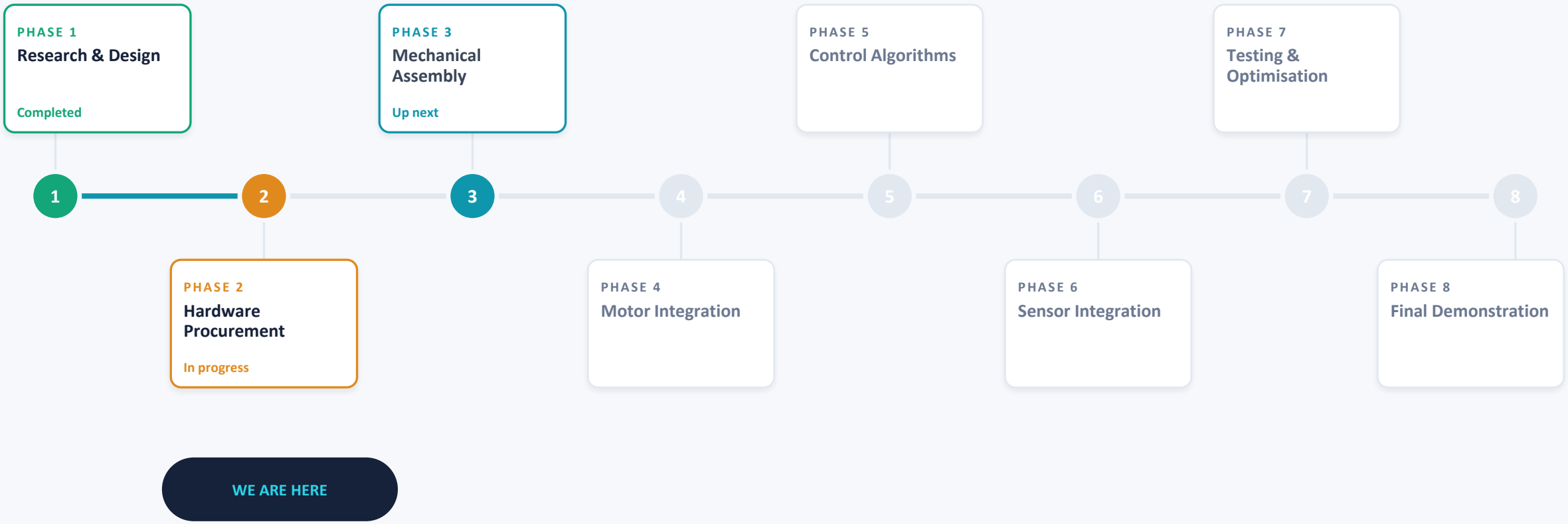


Commit history — tracked iterations & uploads



PROJECT TIMELINE

Eight Phases · Currently at 25%



CURRENT ACHIEVEMENTS

Concrete Wins in Quarter One



Architecture finalized

System block diagram locked.



CAD design completed

Segment modelled to motor specs.



Prototype fabrication started

Two segments printed & checked.



Pi environment established

OS, network and tooling ready.



GitHub workflow operational

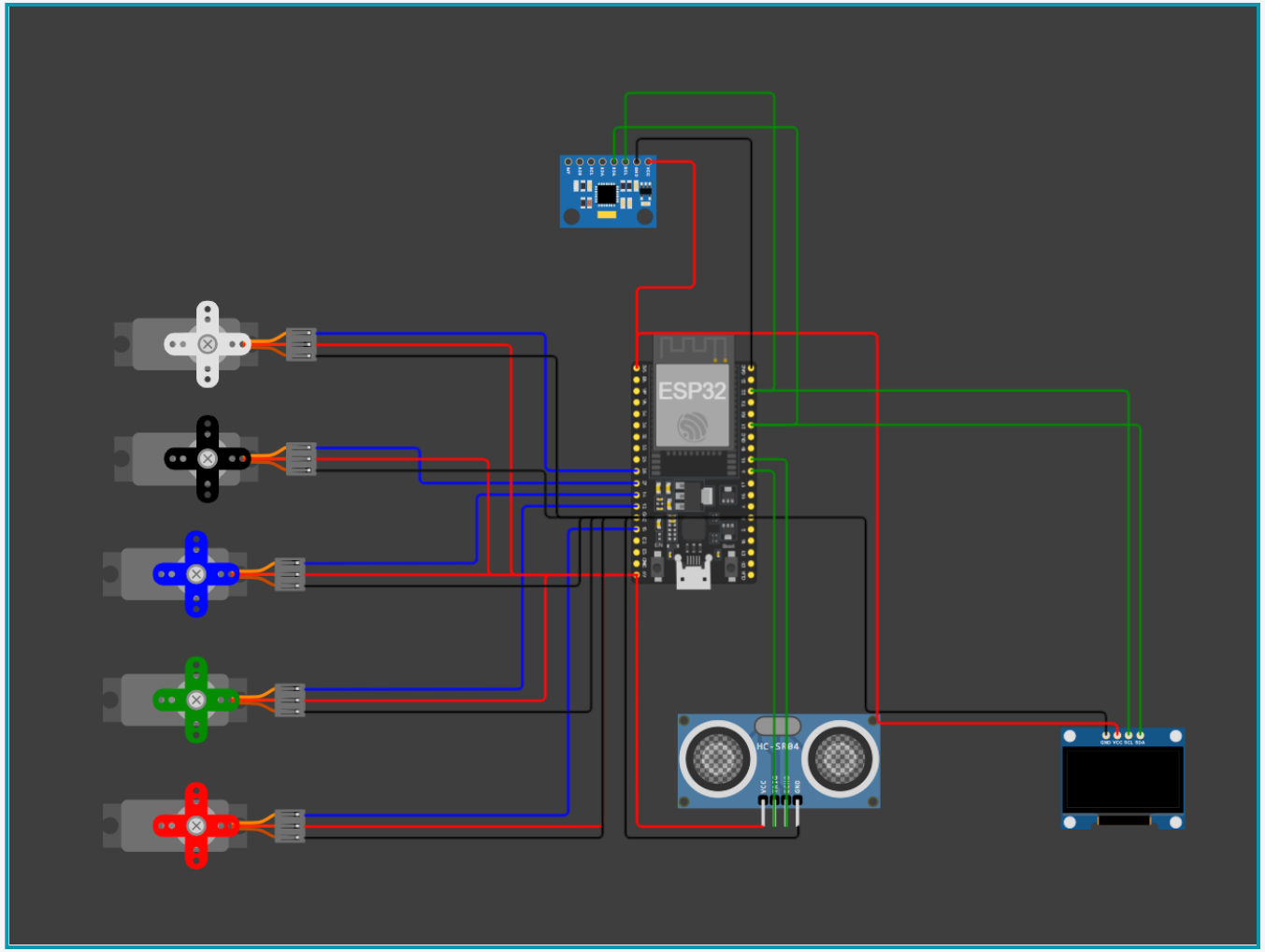
Version control and docs live.



Procurement initiated

Core compute received; rest ordered.

Simulation-Based Validation of Control Architecture



Wokwi-based simulation environment developed to validate locomotion control, sensor integration, and communication architecture before physical Dynamixel hardware arrival.

SIMULATION OBJECTIVE

A virtual testing platform to validate the control architecture before Dynamixel actuators and communication hardware arrive — enabling software development, algorithm verification, and sensor integration in parallel with procurement.

COMPONENTS SIMULATED

Controller | **Sensors** | **Display** | **Actuators**

ESP32 · 5-Servo joint network · MPU6050 IMU · HC-SR04 ultrasonic · SSD1306 OLED · control & communication framework

FUNCTIONAL VALIDATION COMPLETED

- ✓ Multi-joint servo coordination
- ✓ Travelling-wave serpentine gait generation
- ✓ Obstacle detection via ultrasonic sensing
- ✓ IMU integration for orientation feedback
- ✓ OLED-based system status monitoring
- ✓ Sensor–controller–actuator communication

ENGINEERING BENEFITS

- Reduced dependence on hardware delivery timelines
- Enabled early software debugging & control-architecture verification
- Established migration path to Raspberry Pi 5 + Dynamixel XL330, reducing integration risk

KEY OUTCOME A complete software-validation framework was developed, allowing locomotion, sensing, and control workflows to be tested before physical robot assembly.

Planned Activities for the Next Development Phase

1 Hardware Arrival

- Receive Dynamixel XL330 motors
- Receive U2D2 interface
- Verify purchased components

2 Mechanical Assembly

- Assemble complete snake body
- Install Dynamixel actuators
- Route communication cables

3 Communication Setup

- Configure motor IDs
- Establish serial communication
- Validate daisy-chain architecture

4 Locomotion Development

- Port simulation code to Pi 5
- Implement serpentine locomotion
- Synchronize multi-joint movement

5 Sensor Integration

- Integrate MPU6050 IMU
- Integrate obstacle detection
- Implement feedback control

6 Testing & Optimization

- Motion performance testing
- Power & reliability analysis
- System optimization

Completed phase **Ongoing work** **Future milestone**

EXPECTED DELIVERABLES

- Fully assembled snake robot
- Dynamixel-based locomotion
- Sensor-assisted navigation
- Integrated Raspberry Pi 5 control system
- Demonstration-ready prototype
- Final SURGE project showcase

TARGET FOR NEXT REVIEW

- Complete hardware integration
- Demonstrate basic snake locomotion
- Validate communication architecture
- Validate sensing framework

Goal: ~60–70% project completion

WHAT CHANGED SINCE THIS REVIEW

Generation 2 — Corrections & Cause

This slide records what has since been superseded.

	THAT DECK SAID	TODAY	WHY
	XL330 · U2D2 · Power Hub — “ORDERED, AWAITING DELIVERY”	**Order cancelled. 10× ST3215 + Waveshare ESP32 driver in hand**	Part unobtainable in India
	“Eight Phases · Currently at 25%”	**Phase 3 — Fabrication & Assembly**	Eight weeks of Gen 2 work since
	“Prototype fabrication started”	**Gen 1 segments printed; Gen 2 v6 printed**	Design of record changed
	Roadmap step 1 — “Receive Dynamixel XL330 motors”	**Superseded — see slide 29**	Different motors received

Constraint

WHAT HAPPENED

2 of 10 XL330 received, then the part vanished

Six-week stall

IMPACT

Control stack written against absent hardware

Re-architecture

COMMIT a32b675 · 2026-08-20

“No dynamixel in the market. So switched to Servos”

WHAT IT BOUGHT 5.7× joint torque (0.52 → 2.94 N·m) · a 12 V rail · sourceable in India — consequences of the switch, not its reason. Torque budget: slide 23.

The planar robot is working; Gen 3 is next

Gen 2.0

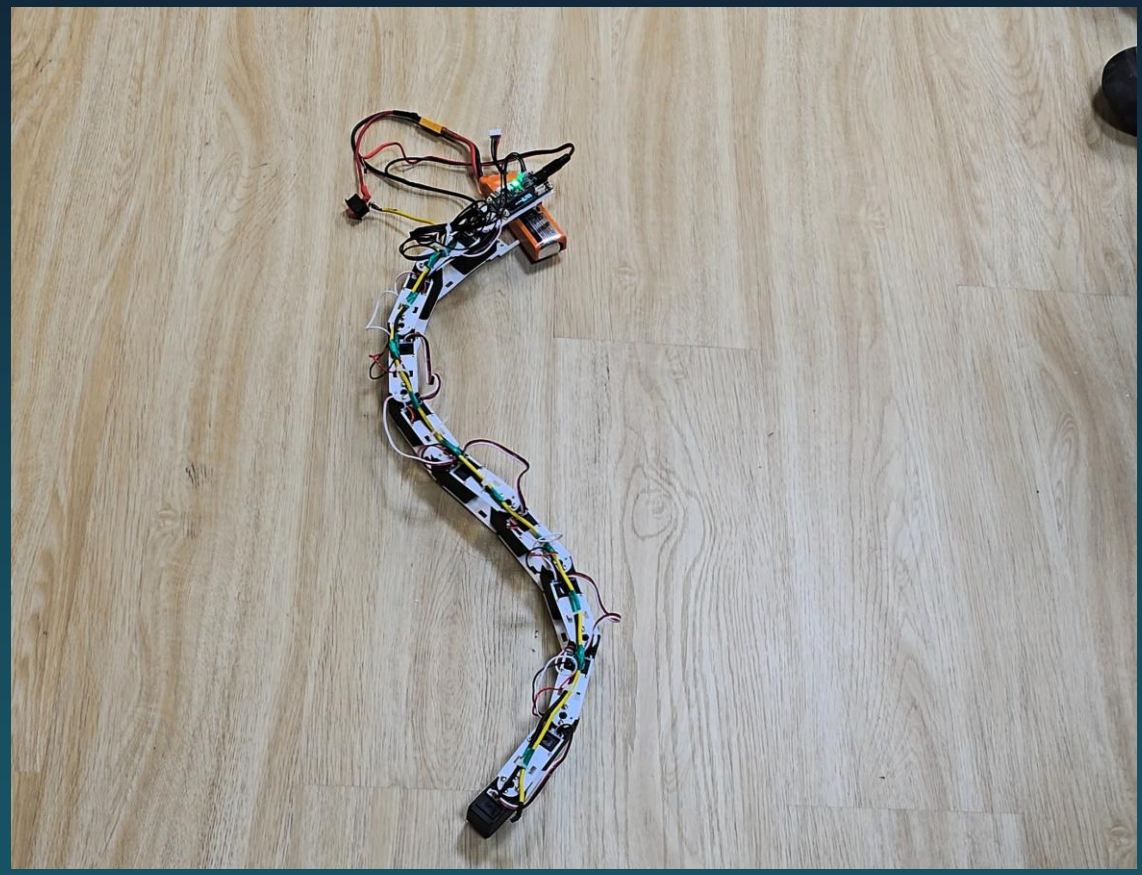
Working planar robot with bench and onboard control.

Gen 2.1

Shared host control, dashboard and gamepad.

Gen 3.0

Alternating yaw and pitch joints in development.



Current physical reference: the ST3215 planar robot.

The actuator changed; the modular idea stayed

Gen 1 · Dynamixel XL330

Ten-joint modular architecture and smart servo feedback.

Gen 2 · Waveshare ST3215

New motor mounts, bus interface and power distribution.

The redesign connected mechanics, wiring and software.



The supply constraint prompted the change; the coordinated-joint concept continued.

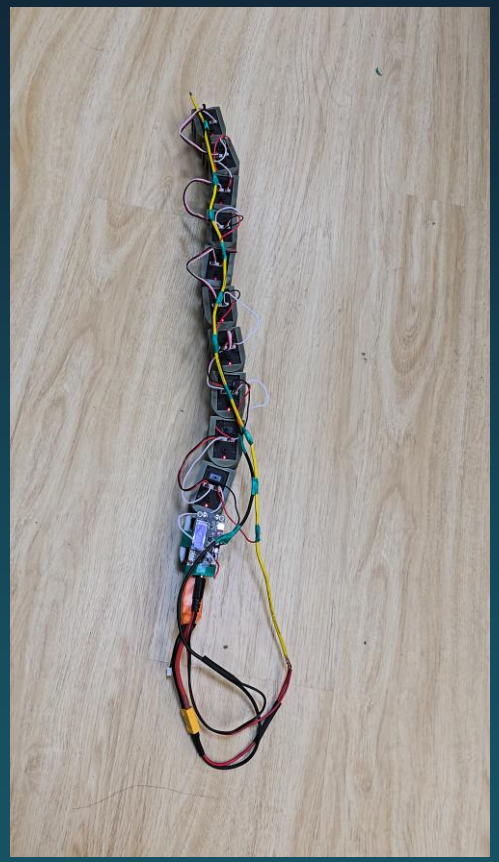
Eleven segments, ten coordinated joints

Modular body

Repeatable sections support the servo chain.

Planar motion

Ten yaw joints receive phase-shifted targets.



Full-chain photograph; architecture labels describe the Gen 2 design.

Ground contact is part of the mechanism

Joint support

Yokes carry the joint load; retention keeps the motor seated.

Directional contact

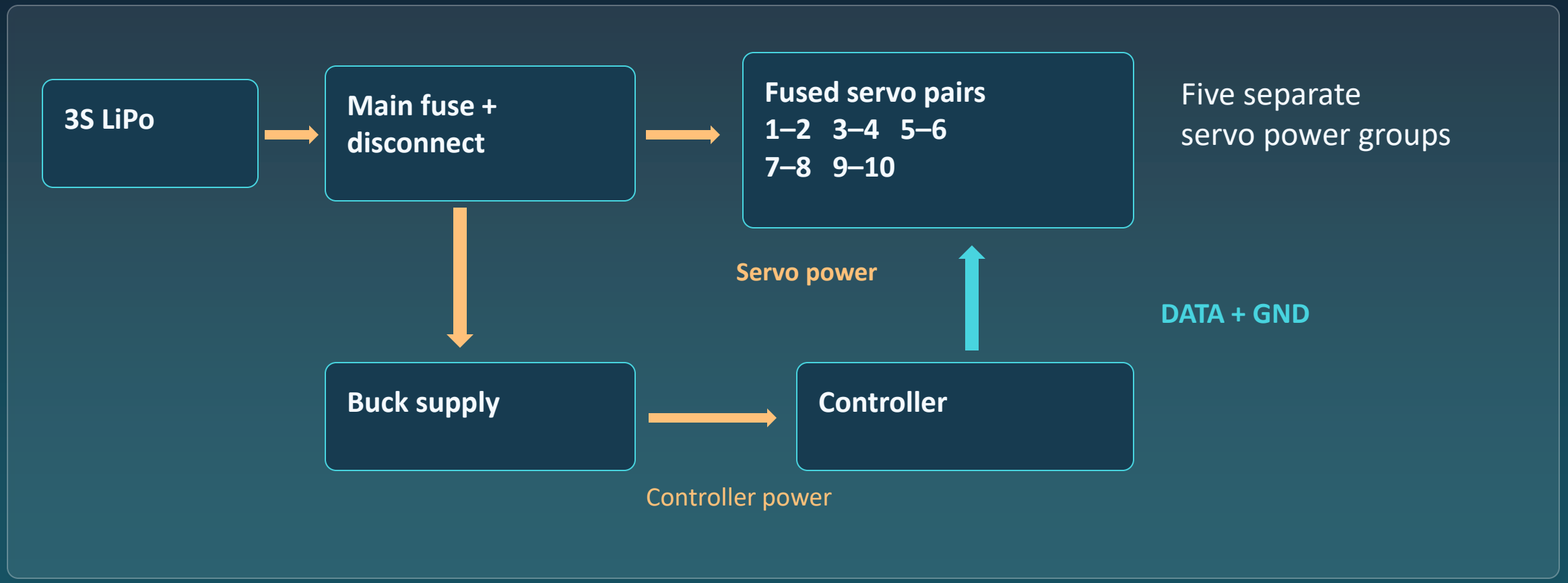
The belly design aims to resist lateral slip while allowing forward travel.

Joint and belly close-up

Photo pending

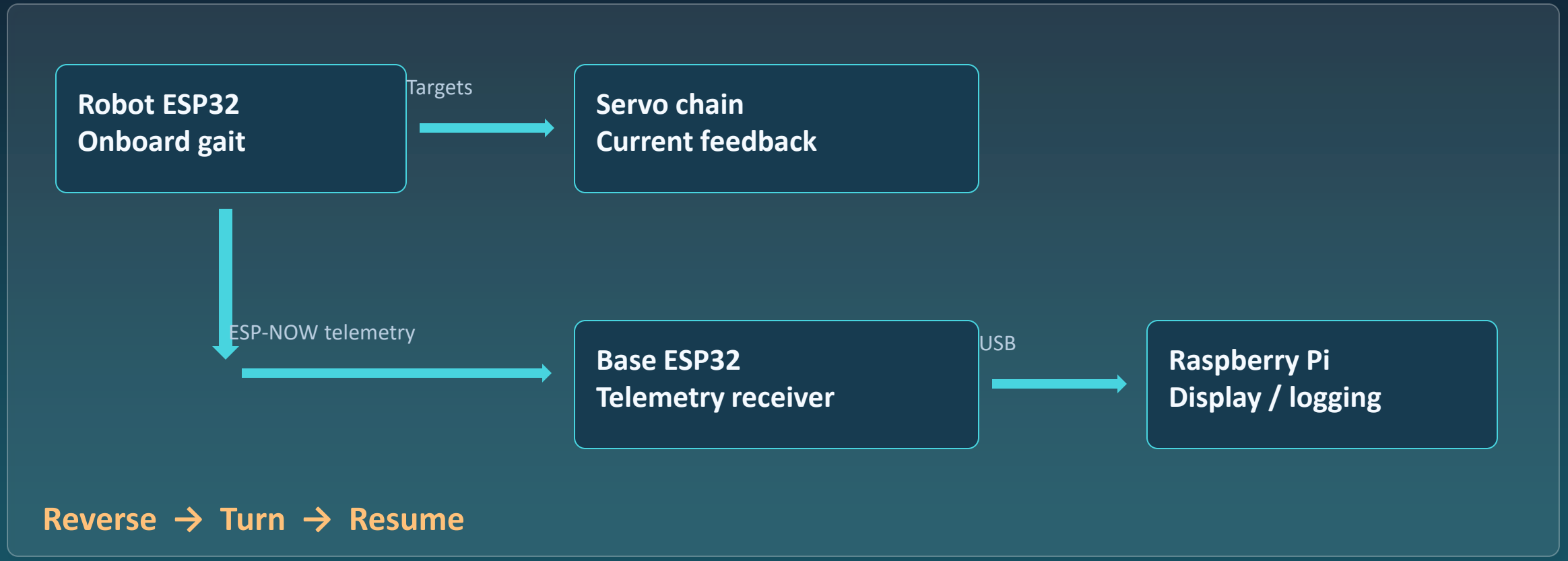
v6 design intent. Traction, motor fit and clearances require physical measurements.

Power reaches the servos in five pairs



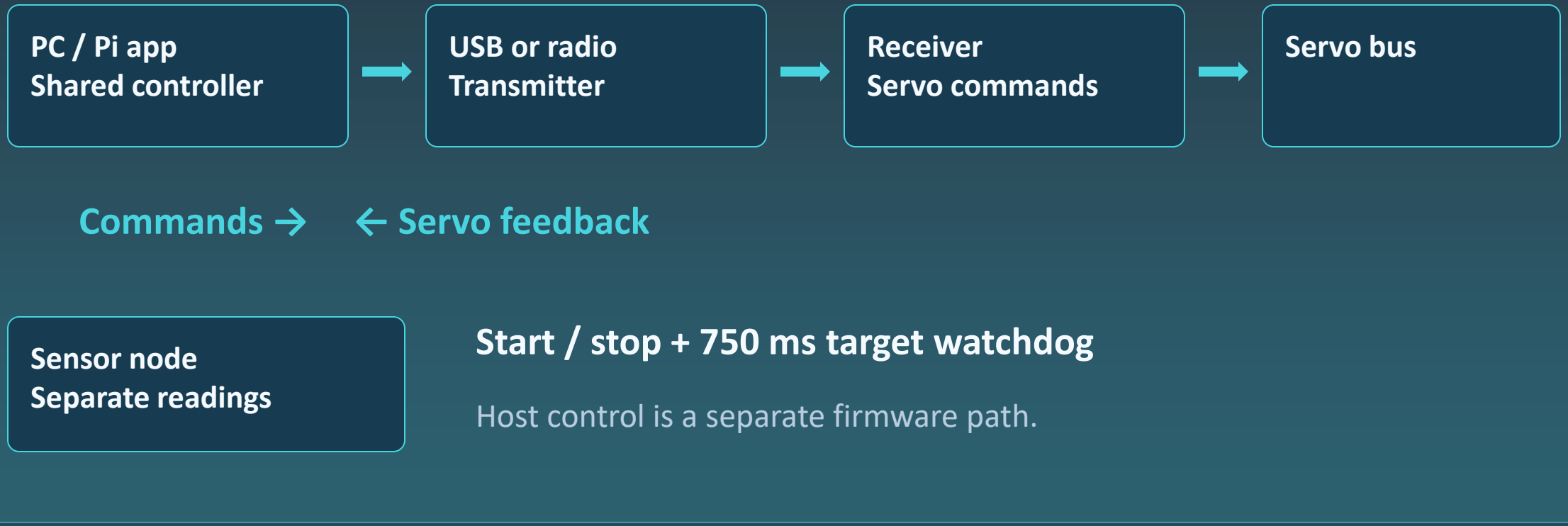
Power design v3: controller supply is separate; shared ground provides the data reference.

Gen 2.0 keeps the gait on the robot



Stall thresholds require calibration; current sign alone does not locate an obstacle.

Gen 2.1 moves target generation to the host

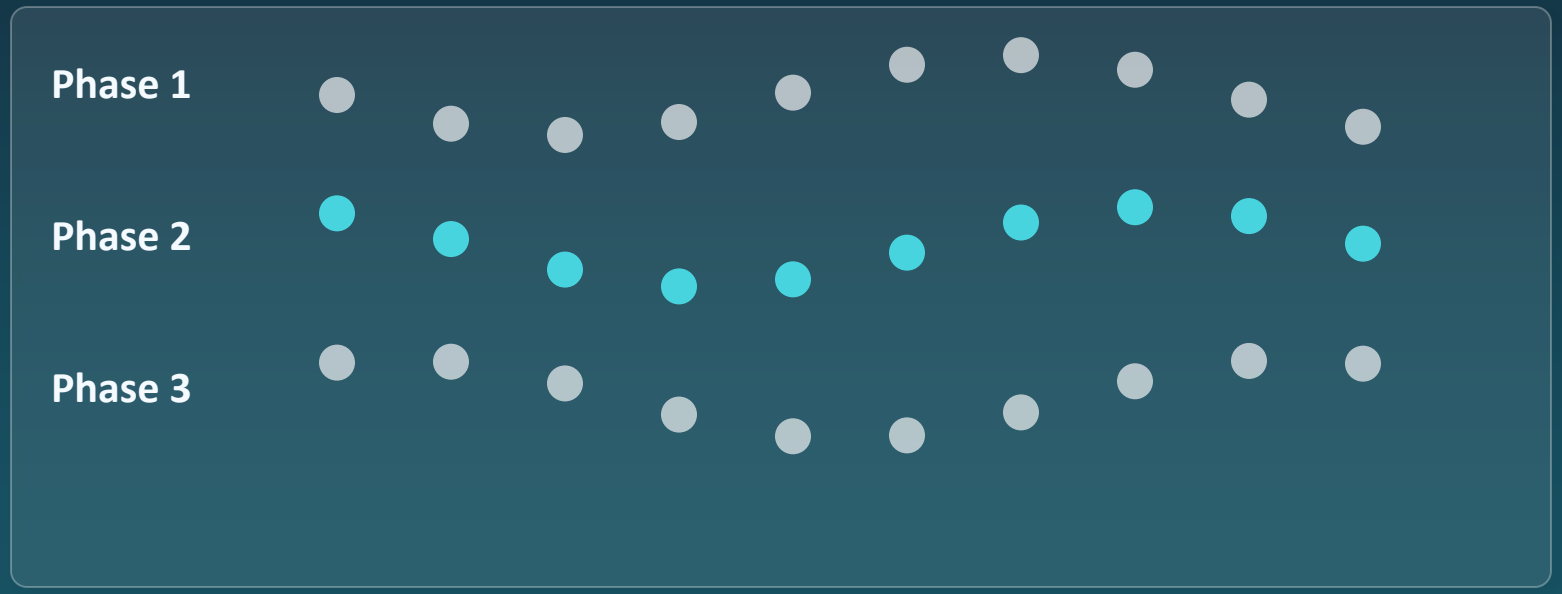


Gen 2.1 does not inherit the onboard Gen 2.0 battery/temperature stops or evasion logic.

A travelling wave coordinates the joints

$$\theta_i(t) = A \sin(\omega t - i\phi) + b$$

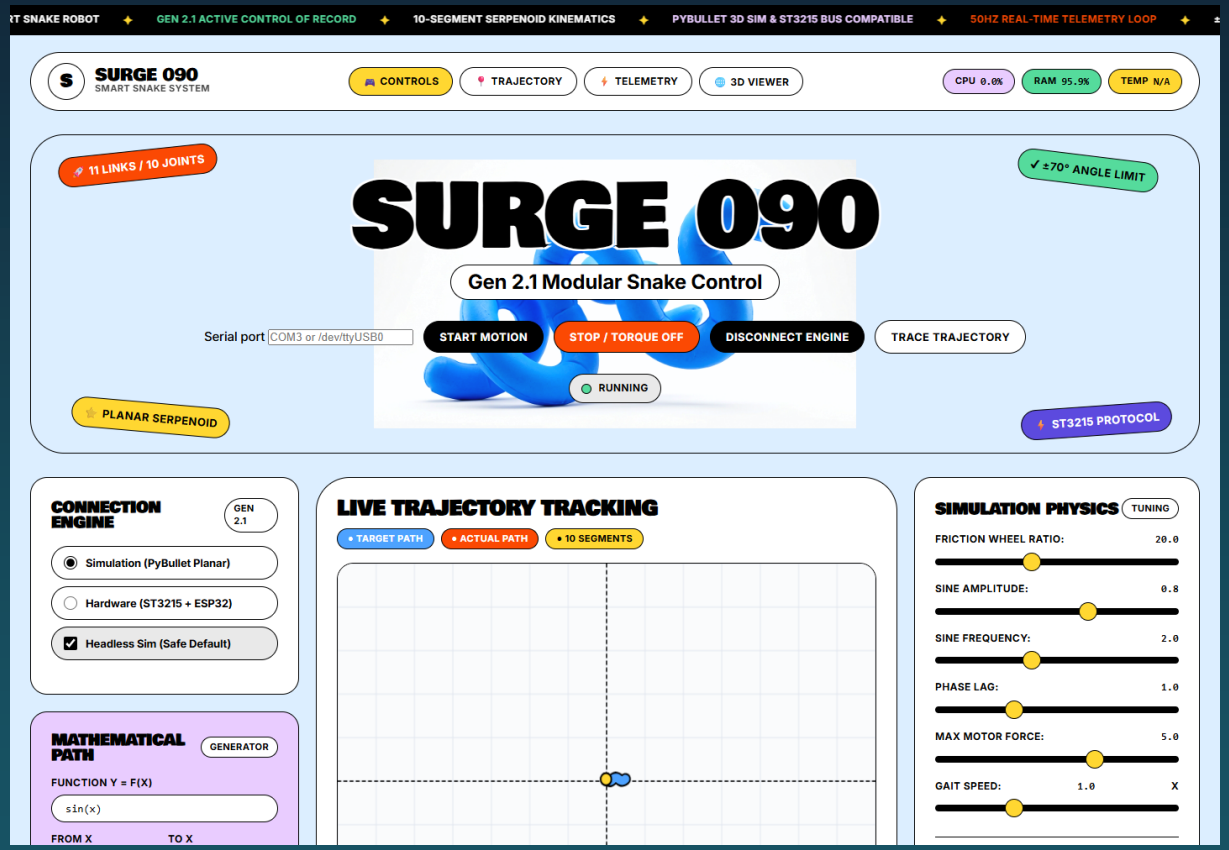
A amplitude · ω frequency · ϕ phase lag · b steering bias





Controller illustration, not a measured trajectory. Different firmware paths use different defaults.

One controller, several ways to work with it



Dashboard

Tune gait and read feedback.

Desktop

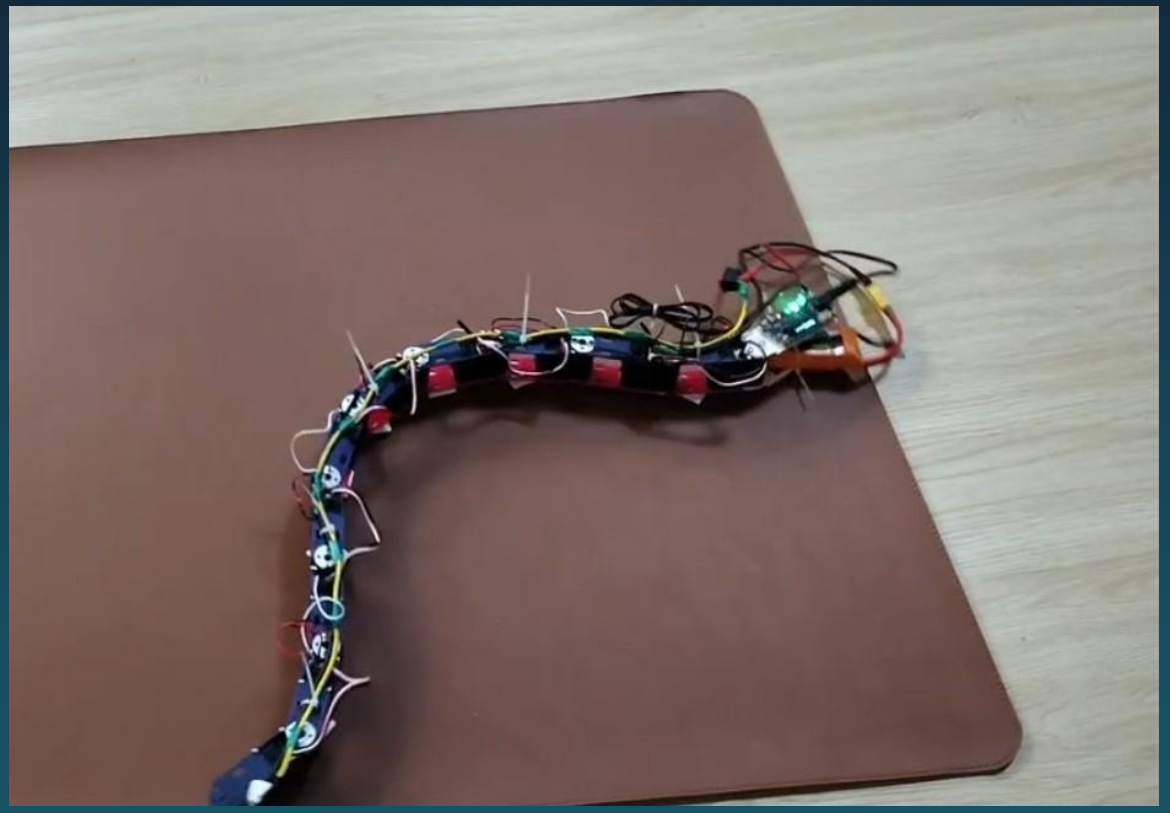
GUI or headless simulation.

Gamepad

Manual steering and start/stop.

Captured in SIM mode. Paths and accuracy reports require simulation or future hardware localization.

What we can demonstrate now



Locomotion video to be added

Still photograph from project trials. No measured trajectory or performance values are inferred.

Working operation

Gen 2 motion and control interfaces.

Next measurements

Speed, slip, power use and stop latency.

Gen 3 introduces alternating yaw and pitch

Alternating joint-axis concept



Revised connectors and plate-chain layout

Experimental waveform; physical clearance and body support remain open work.

Concept schematic, not CAD or a completed Gen 3 build. The present scripts default to dry run.

The next experiments answer specific questions

EXPERIMENT	OBSERVATIONS TO RECORD
Surface contact and forward travel	Distance, time and lateral slip
Steering and stop response	Turn response and stop delay
Pitch joints and body support	Clearance, contact and current

Use a recorded controller configuration and test surface. Enter values only after measurement.

From the planar robot to Gen 3 trials

01

Working Gen 2

Reference platform

→

02

Measure motion

Motion, contact, power

→

03

Check Gen 3 assembly

Axes and clearances

→

04

Run 3D trials

Coordinated yaw / pitch

A working planar base for the next mechanical and gait experiments.

Shashank Jangir · Bhavesh Bansiwal · Mahima Chotiya